

Desarrollo Asistido por IA en Entornos Empresariales

Gobernanza de IA con GitHub Copilot y Spec-Driven Development

Cristiam Espinoza

Principal Consultant Data & AI

cespinoz@microsoft.com





Cristiam Espinoza

Principal Consultant • Microsoft

Especializado en Arquitectura de Datos, Inteligencia Artificial, Ingeniería de Datos y DevOps.

Ha liderado iniciativas de modernización tecnológica para organizaciones de los sectores financiero, manufactura, consumo masivo y gobierno en Latinoamérica.

/constitution

/specify

/clarify

/plan

/tasks

/analyze

/implement

EL PROBLEMA EN LA ERA DE LOS AGENTES

El riesgo no es que el código sea malo.

Es que nadie sabe por qué se tomaron las decisiones.



Decisiones sin autor

El agente eligió una librería, un patrón de retry, un esquema. Nadie sabe si fue criterio o azar del prompt.



Contexto que muere

El razonamiento vivía en un hilo de chat. Se cierra la sesión y el equipo hereda código sin el porqué.



Estándares por suerte

Cinco desarrolladores, cinco agentes, cinco arquitecturas. Todas funcionan. Ninguna es compatible entre sí.



*En una auditoría, «lo generó Copilot» no es una respuesta.
La velocidad sin trazabilidad es deuda técnica que todavía no sabes que tienes.*





EL CAMBIO DE PARADIGMA

De prompting ad-hoc a Spec-Driven Development

Prompting ad-hoc

- La intención vive en el chat, no el repo.
- Generación de una sola pasada: se acepta o se “repromptea”.
- El código es la única fuente de Verdad.
- Cada desarrollador improvisa sus propias reglas.
- Revisión humana al final, sobre el diff.

Spec-Drive Development

- La intención vive en el repo, versionada y revisable.
- Refinamiento multi-etapa antes de escribir código.
- La spec es la fuente de la verdad, el código es su salida.
- Principios organizacionales aplicados por gates.
- Revisión humana en cada artefacto, antes del código.



La especificación deja de ser andamiaje desechable: se vuelve ejecutable y genera la implementación, en lugar de solo orientarla.



EL TOOLKIT

GitHub Spec Kit ♦ Hyper Velocity Engineering

Spec Kit es el toolkit open source de GitHub (MIT) que materializa SDD: un CLI llamado specify, más un conjunto de prompts y plantillas que se instalan dentro de tu repositorio.

Bootstrap del proyecto

```
uv tool install specify-cli
specify init mi-proyecto --integration copilot
# → .specify/ · slash commands · plantillas
```

HVE (Hyper Velocity Engineering) es la capa de mentalidad y componentes reutilizables — instrucciones, prompts, agentes y skills de Copilot — que Microsoft publica en hve-core. Spec Kit aporta el proceso; HVE aporta el arsenal.



Más de 30 agentes soportados

Copilot, Claude Code, Cursor, Codex CLI, Gemini CLI, Windsurf. Los artefactos son markdown: cambiar de agente no reescribe tus specs.



Slash commands en tu IDE

/speckit.constitution, /speckit.specify, /speckit.plan... inyectados en el proyecto y ejecutados por el agente que ya usas.



Diseñado para restricciones enterprise

Estándares organizacionales, design systems y requisitos de cumplimiento como parte del proceso, no como un PDF aparte.



EL CIMIENTO

constitution.md – las reglas no negociables

Se escribe una sola vez por proyecto, antes de cualquier feature. Es lo que mantiene arquitectónicamente compatibles a varios desarrolladores y varios agentes trabajando en paralelo.

Artículo I Ninguna feature empieza sin spec aprobada

Artículo II No hay código de implementación antes de las pruebas

Artículo III Todo acceso a datos pasa por la capa de contratos

Artículo IV Simplicity gate: máximo N proyectos por feature



Por qué gobierna de verdad

- No es una guía de estilo: es un contrato que el agente lee en cada fase
- Se aplica mediante phase gates — el plan técnico debe pasar el constitution check
- Vive en el repositorio: se discute en un PR, se versiona, se audita en el historial
- Distintos agentes producen código que sigue los mismos principios



Cuatro artefactos en markdown = una traza auditable



constitution.md

Principios de gobierno del proyecto

Quién revisa: arquitectura + seguridad.

Cambia rara vez; cada cambio es una decisión de gobierno.



spec.md

Requisitos e historias de usuario

Quién revisa: producto + negocio.

El qué y el por qué, sin decisiones de stack.



plan.md

Diseño técnico y decisiones

Quién revisa: tech lead.

Stack, modelo de datos, contratos y el resultado del constitution check.



tasks.md

Lista ordenada y con dependencias

Quién revisa: el equipo de desarrollo.

Unidades ejecutables, opcionalmente exportadas a Issues.



Viven en el repositorio: entran por pull request, se comentan, se versionan y se recargan como contexto persistente en cada sesión del agente.

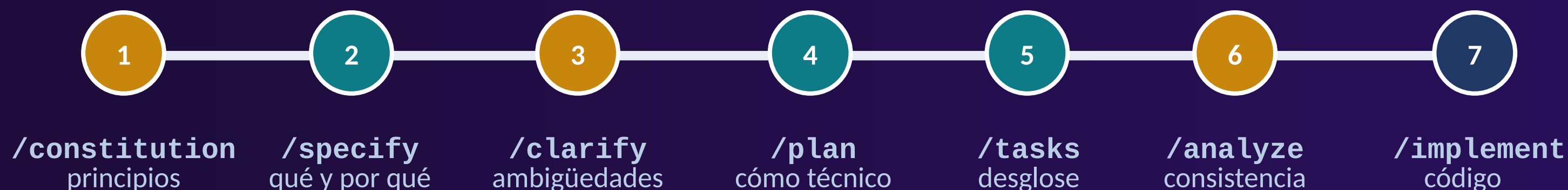


EL FLUJO

Siete pasos: el equipo dirige, el agente implementa

REVISIÓN HUMANA OBLIGATORIA EN CADA ARTEFACTO

EJECUCIÓN DEL AGENTE



Comandos complementarios

`/speckit.checklist`

Checklists de calidad que validan que los requisitos estén completos y sin ambigüedad — «pruebas unitarias para el lenguaje natural».

`/speckit.taskstoissues`

Convierte el desglose de tareas en GitHub Issues: el trabajo del agente queda rastreable en el mismo tablero del equipo.

`/speckit.converge`

Evalúa el código real contra spec, plan y tasks, y añade como tareas nuevas lo que quedó pendiente o se desvió.



LA EVIDENCIA

Cuatro artefactos en markdown = una traza auditable



constitution.md

Principios de gobierno del proyecto

Quién revisa: arquitectura + seguridad. Cambia rara vez; cada cambio es una decisión de gobierno.



spec.md

Requisitos e historias de usuario

Quién revisa: producto + negocio. El qué y el por qué, sin decisiones de stack.



plan.md

Diseño técnico y decisiones

Quién revisa: tech lead. Stack, modelo de datos, contratos y el resultado del constitution check.



tasks.md

Lista ordenada y con dependencias

Quién revisa: el equipo de desarrollo. Unidades ejecutables, opcionalmente exportadas a Issues.



Viven en el repositorio: entran por pull request, se comentan, se versionan y se recargan como contexto persistente en cada sesión del agente.

¿CÓMO EMPEZAR?



Una feature, una constitución real, el ciclo completo

1 Elige una feature acotada y real

Nada de un hola mundo, pero tampoco el core del negocio. Algo cuyo resultado el equipo pueda juzgar.

2 Escribe una constitution honesta

Cuatro o cinco artículos que ya seas capaz de defender en un code review. No copies plantillas genéricas.

3 Recorre el flujo de punta a punta

constitution → specify → clarify → plan → tasks → analyze → implement. Sin saltarte los gates la primera vez.

4 Mide sobre tu propio trabajo

Compara retrabajo, claridad del PR y tiempo de onboarding — no la promesa de nadie.

Qué esperar, sin adornos

- El overhead de la spec puede superar su beneficio en cambios triviales
- En brownfield multi-módulo la estructura tiende a producir volumen, no fidelidad
- El CLI evoluciona rápido: tutoriales antiguos muestran flags que ya no existen

 La spec es el nuevo artefacto de gobierno.
El código es su consecuencia.



Gracias

Conectemos

Información de contacto

Email: cespinoz@microsoft.com

Linkedin:
<https://www.linkedin.com/in/cristiam-espinoza-malpartida-8815763/>

Información de contacto

Slides:

Code Examples: github.com/devops-
Join the conversation: #DODLima #DevOpsDaysLima